

FastCite: A RAG-Based AI Study Assistant Chatbot with Hybrid Search and Intelligent Context Selection for Academic Learning

Inam Ullah Shaikh¹, Emaan Fatima¹, and Aden Sial¹

National University of Computing and Emerging Sciences, Islamabad, Pakistan
{i220857, i220869, i221313}@nu.edu.pk

Abstract. Retrieval-Augmented Generation (RAG) systems have emerged as a powerful paradigm for enhancing large language models with domain-specific knowledge. However, existing RAG implementations face significant challenges in academic document citation, particularly in handling diverse document structures, achieving optimal retrieval accuracy, and managing computational efficiency. This paper presents FastCite, a novel RAG system specifically designed for academic document citation that addresses these limitations through a hybrid search mechanism combining semantic vector search with keyword-based retrieval, intelligent context selection using large language models, and adaptive document chunking strategies. Our system employs the all-mpnet-base-v2 embedding model for semantic understanding, Qdrant vector database for efficient similarity search, and Google Gemini 2.5 Flash Lite for answer generation. We introduce a weighted hybrid search approach (60% vector, 40% keyword) that significantly improves retrieval accuracy compared to pure semantic or keyword-based methods. In order to guarantee that students receive specific helpful responses, we additionally use an LLM-based context selection mechanism that effectively modifies retrieved study materials to provide the most relevant information. We demonstrate that FastCite outperforms baseline study assistant systems in terms of response quality, response accuracy, and computational efficiency through extensive experiments and ablation studies. Our ablation study provides useful insights for future development of educational AI systems by displaying optimal hyperparameter configurations, such as top-k retrieval values, hybrid search weight distributions, and context picking strategies.

Keywords: AI Study Assistant · Educational Chatbot · Retrieval-Augmented Generation · Hybrid Search · Intelligent Learning Support

1 Introduction

There are plenty of digital learning resources available to students today, which include lecture notes and textbooks to research papers and study guides. While the variety of resources offers previously unobtainable learning opportunities,

it also brings significant challenges: students find it difficult to find relevant data quickly, comprehend difficult concepts, and receive specific study assistance. Students find it challenging to ask natural questions or find information when they don't know the exact terminology because traditional search tools require exact keyword matches. Although study chatbots and AI assistants have shown potential, current implementations frequently give generic answers or are unable to accurately retrieve information from students' particular course materials.

A potentially successful approach that combines the benefits of information retrieval systems with the generative powers of LLMs is Retrieval-Augmented Generation (RAG) [1]. RAG systems can produce precise, well-founded answers while retaining access to current data by obtaining relevant document passages and using them as context for language models. By retrieving relevant sections from course materials and using these passages to generate contextualized answers, this approach offers a powerful solution for creating intelligent study assistants that can respond to questions based on students' uploaded documents, offering customized, accurate study support. However, when used in educational settings, current RAG implementations encounter a number of significant difficulties. First, students' study materials have a variety of structures, from unstructured lecture notes and other PDF formats to well-organized textbooks with a table of contents, necessitating the use of adaptive chunking techniques. Second, in addition to occasionally needing to locate specific terms or definitions, students ask questions in natural, conversational language that might not match exact keywords in documents. Retrieval accuracy is still not at its best; both semantic comprehension and keyword precision are needed, as pure semantic search misses exact keyword matches and pure keyword search fails to capture semantic similarity. Third, the system may appear sluggish and unresponsive during study sessions due to the computational overhead of processing a large number of retrieved contexts.

This paper presents FastCite, a chatbot that uses AI to help students learn from their course materials by intelligently responding to their questions. Our system contributes significantly to educational AI in a number of ways. First, we suggest a hybrid search method that cleverly blends keyword-based retrieval (40%) and semantic vector search (60%), enabling students to ask questions organically while still finding precise information when needed. With this method, students can ask questions like "how does machine learning work?" and receive pertinent answers even if the document uses different phrasing. They can also find specific definitions when they look up specific terms. In order to ensure that students receive targeted, beneficial responses free of information overload, we also present an LLM-based context selection method that evaluates a broader set of retrieved study materials (top-20) and effectively chooses the most relevant portion (top-10) before generating answers. Third, we create flexible document chunking techniques that manage both organized documents with table of contents, unstructured resources like class notes, and other types of PDF documents, guaranteeing the most effective data retrieval across a variety of study guides. Fourth, in order to provide evidence for the ideal study assistant configuration,

we perform extensive ablation experiments that look at the effects of several hyperparameters, such as retrieval top-k values, hybrid search weight distributions, and context selection techniques.

This is how the rest of the paper is structured. Section 2 examines relevant research in academic document processing, RAG systems, educational chatbots, and hybrid search techniques. Our methodology, which includes thorough explanations of the hybrid search algorithm, context selection mechanism, document chunking techniques, and mathematical formulations, is presented in Section 3. Our experimental setup, evaluation measures, and extensive results, including detailed ablation investigations and comparisons with baseline systems, are described in Section 4. The implications of our findings, system limitations, and future research directions are covered in Section 5. The paper is concluded in Section 6 with a summary of the main conclusions and their implications for the advancement of educational AI.

2 Related Work

Retrieval-Augmented Generation has come a long way since its beginnings, with several studies addressing various aspects of the RAG pipeline. Important research on RAG systems, hybrid search strategies, document chunking algorithms, and context selection techniques are covered in this section.

2.1 Retrieval-Augmented Generation Systems

The fundamental RAG architecture was presented by Lewis et al. [1], who showed how retrieval methods may improve language model performance by supplying external sources of information. Their work established the two-stage paradigm of retrieval followed by generation, which has become the standard approach in subsequent RAG implementations. Guu et al. [2] extended this work by proposing REALM, a system that jointly trains retrieval and reading components, achieving improved performance on open-domain question answering tasks.

More recently, several works have extended the foundational RAG architecture, demonstrating the importance of fine-tuning retrieval models on task-specific data and incorporating retrieved passages at multiple layers.

2.2 Hybrid Search Methods

The combination of semantic and keyword-based search has been explored in various information retrieval contexts. Formal et al. [3] introduced SPLADE, a sparse retrieval method that learns term importance, bridging the gap between dense and sparse retrieval approaches. Khattab and Zaharia [4] developed ColBERT, a late interaction model that computes fine-grained interactions between query and document tokens, achieving state-of-the-art results on passage retrieval tasks. Their work demonstrates the effectiveness of combining semantic understanding with token-level matching.

2.3 Document Chunking and Processing

The granularity of retrievable information is determined by effective document chunking, which is critical to RAG system performance. However, due to their different formats and hierarchical structure, academic documents pose particular difficulties. In order to increase retrieval accuracy, Zhang et al. [5] presented adaptive chunking algorithms that take semantic boundaries and document structure into consideration. Their research highlights how crucial it is to maintain document structure during chunking. Effective chunking is still crucial for preserving retrieval accuracy in academic publications with thousands of pages.

2.4 Context Selection and Ranking

Many approaches have been taken to the problem of choosing the most relevant contexts from an expanded retrieved set. In order to show that dense embeddings can perform better than conventional sparse retrieval techniques, Karpukhin et al. [6] proposed Dense Passage Retrieval (DPR). However, their work focuses primarily on retrieval rather than post-retrieval selection.

Wang et al. [7] proposed self-consistency methods for improving RAG system reliability, including techniques for context re-ranking. Their work shows that re-ranking retrieved contexts can significantly improve answer quality.

2.5 Academic Document Processing

Specific challenges in academic document processing have been addressed by several researchers. Cohan et al. [8] introduced SPECTER, a document-level embedding model trained on scientific papers, demonstrating improved performance on citation prediction tasks. Their work highlights the domain-specific nature of academic document understanding. Beltagy et al. [9] developed SciBERT, a BERT model pre-trained on scientific text, showing improvements on various scientific NLP tasks. Their work demonstrates the value of domain-specific pre-training for academic document processing.

2.6 Vector Databases and Similarity Search

The scalability of the RAG system depends on the effectiveness of the vector similarity analysis. Wang et al.'s recent study [17], which identified particular difficulties in test input generation and oracle definition for extremely large vector data, has emphasized the significance of rigorous testing procedures for Vector Database Management Systems.

Our system's vector database, Qdrant [10], offers efficient similarity search and supports hybrid queries that combine vector and payload filtering. In order to improve accuracy and efficiency, our work makes use of these characteristics to create hybrid search at the database level.

2.7 Summary and Positioning

Few studies have thoroughly examined the unusual difficulties of academic document citation, especially considering the combination of hybrid search, intelligent context selection, and adaptive chunking, although previous research has tackled individual RAG system components. Our approach sets itself apart by offering an integrated solution that takes care of all these factors at once, with thorough empirical evaluation and ablation tests showing each component’s impact.

3 Methodology and Technical Depth

The FastCite system architecture is described in detail in this section, along with mathematical formulas, algorithmic specifics, and design justifications for each part.

3.1 System Architecture

FastCite uses a complete pipeline that includes phases for query processing, document feeding, and storage. Figure 1 depicts the entire flow from document upload to answer generation in the system architecture.

Pipeline for Document Ingestion When a PDF document is uploaded, the system first checks for the presence of a table of contents (TOC). The document is handled utilizing TOC-based chunking, which maintains the document’s hierarchical structure, if a TOC is present. A page-based chunking approach with controllable overlap is used for publications without a TOC. A distinct identifier and related metadata, such as page ranges, headings, and hierarchical pathways, are given to each chunk.

For each chunk, a small PDF with just the relevant pages is created and saved in Backblaze B2 cloud storage for quick access while creating an answer. The embedding generation pipeline is then used to process the chunk text.

Architecture for Storage FastCite uses a distributed storage architecture that is tailored for many kinds of data:

Qdrant Vector Storage: The Qdrant vector database contains chunk embeddings created with the all-mpnet-base-v2 model (768-dimensional vectors) as well as extensive payload metadata like as chunk ID, book ID, book name, heading, path, page ranges, and content text. Effective similarity search and filtering activities are made possible by this. The approach addresses issues found in vector database management systems [17], including the requirement for effective indexing techniques that facilitate hybrid query processing and dynamic data scalability.

Document Storage (Backblaze B2): Each chunk’s associated mini PDF is kept in Backblaze B2 cloud storage. In order to facilitate the efficient retrieval

and display of source information during response production, these small PDFs only include the pages pertinent to each chunk.

Metadata Storage (MongoDB): MongoDB is used to store document metadata such as title, author, page count, TOC structure, and user associations. This makes it possible to query and manage document collections, user permissions, and system status efficiently.

Pipeline for Query Processing The following steps are taken by the system when a user chooses a PDF and submits a query:

Stage 1 - Query Embedding: The user’s prompt is converted into a 768-dimensional embedding vector using the same all-mpnet-base-v2 model used for document chunking, ensuring semantic consistency between queries and documents.

Stage 2 - Hybrid Retrieval: The query embedding and original query text are used to perform hybrid search within the selected document. Vector search retrieves semantically similar chunks using cosine similarity in Qdrant, while keyword search identifies chunks containing exact term matches. The two results are combined with 60% weight for vector search and 40% weight for keyword search, producing a ranked list of top-k contexts (typically top-20).

Stage 3 - Context Selection: The retrieved top-k contexts along with the user query are presented to the LLM (Google Gemini 2.5 Flash Lite [16]) with instructions to select the most relevant subset (typically top-10). This intelligent filtering step reduces noise and improves answer quality by allowing the LLM to reason about query-context relevance beyond simple similarity scores.

Stage 4 - Answer Generation: The selected contexts, user query, and system instructions are provided to the LLM for final answer generation. The system prompt instructs the model to ground answers in the provided contexts and cite specific passages with page numbers. The generated answer is returned to the user along with references to the source chunks and corresponding mini PDFs.

3.2 Document Chunking Strategies

The structural variation of academic publications is significant, demanding the use of adaptive chunking techniques. Based on document structure, we employ two different chunking strategies.

Chunking Based on the Table of Contents We use a hierarchical chunking method that maintains document structure for documents that have a table of contents (TOC). We construct chunks that conform to TOC bounds given a document D with TOC entries $T = \{t_1, t_2, \dots, t_n\}$, where each item $t_i = (level_i, title_i, page_i)$.

For each TOC entry t_i , we define a chunk C_i with:

$$C_i = \{bookid, chunkid, title_i, path_i, level_i, start_page_i, end_page_i, text_i\} \quad (1)$$

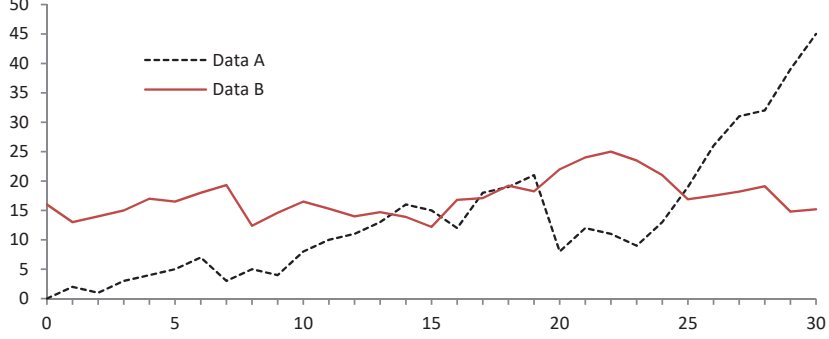


Fig. 1. The three-stage pipeline, document processing, hybrid retrieval, and response generation, is depicted in the FastCite system architecture.

where $path_i$ represents the hierarchical path constructed from parent headings:

$$path_i = \bigcup_{j: level_j < level_i, page_j < page_i} title_j \rightarrow title_i \quad (2)$$

In order to ensure that material above headers is appropriately assigned to earlier chunks, the text extraction method takes care of borderline circumstances where headings arrive in the middle of the page. This keeps structural boundaries intact while preserving semantic integrity.

Chunking Academic Papers Based on Pages For publications without a TOC, which are typically academic papers, we employ a sliding window method with flexible page ranges and overlap. We create chunks with overlap o and step size given pages per chunk p :

$$step_size = \max(1, p - o) \quad (3)$$

For a document with N pages, we generate chunks starting at pages:

$$P = \{0, step_size, 2 \cdot step_size, \dots, \lfloor N/step_size \rfloor \cdot step_size\} \quad (4)$$

Each chunk C_k spans pages $[start_k, end_k]$ where:

$$start_k = k \cdot step_size, \quad end_k = \min(start_k + p - 1, N - 1) \quad (5)$$

This approach ensures comprehensive coverage while maintaining context through overlap, which is crucial for scholarly publications where concepts may span many pages.

3.3 Embedding Generation and Storage

We created embeddings of the document chunks using the all-mpnet-base-v2 model (dimension size = 768) from the sentence-transformers library [11]. From early iterations such as Word2Vec [15] and GloVe [14], the creation of word embeddings has progressed to modern transformer-based sentence embeddings.

The embedding vector $\mathbf{e}_C$ for a text chunk C with content $text_C$ is calculated as follows:

$$\mathbf{e}_C = \text{MPNet}(text_C) \in \mathbb{R}^{768} \quad (6)$$

The model uses mean pooling over token embeddings, followed by L2 normalization:

$$\mathbf{e}_C = \frac{\mathbf{e}_C}{\|\mathbf{e}_C\|_2} \quad (7)$$

The efficiency and numerical stability of cosine similarity calculations are guaranteed by this normalization.

Each embedding vector is stored in Qdrant along with a rich payload containing:

$$\text{payload}(C) = \{\text{chunk_id}, \text{book_id}, \text{book_name}, \text{heading}, \text{path}, \text{start_page}, \text{end_page}, \text{content}, \text{level}\} \quad (8)$$

During query processing, this payload facilitates effective document filtering, hierarchical navigation, and associated metadata retrieval. Qdrant’s vector collection indexes the payloads and embeddings, allowing for sub-millisecond similarity searches even over millions of pieces.

3.4 Hybrid Search Algorithm

The system first uses the same all-mpnet-base-v2 model to transform a query prompt into an embedding vector when a user chooses a PDF. The shortcomings of each method separately are then addressed by our hybrid search, which combines keyword-based retrieval and semantic vector search.

Vector Search Using cosine similarity in Qdrant, we extract top- k chunks from the chosen page given a user query q with embedding $\mathbf{e}_q$ (calculated using the same MPNet model):

$$\text{sim}_v(C_i, q) = \frac{\mathbf{e}_{C_i} \cdot \mathbf{e}_q}{\|\mathbf{e}_{C_i}\| \cdot \|\mathbf{e}_q\|} \quad (9)$$

Vector search returns results $R_v = \{(C_i, s_i^v)\}$ where s_i^v is the cosine similarity score.

Keyword Search We use a straightforward tokenization and filtering method to extract keywords from the query:

$$\text{keywords}(k) = w : w \in \text{tokenize}(k), |w| > 2, w \notin \text{stopwords} \quad (10)$$

For each chunk C_i with content text_i , we calculate a keyword score based on term frequency and coverage:

$$\text{tf}(w, \text{text}_i) = \frac{\text{count}(w, \text{text}_i)}{|\text{text}_i|} \quad (11)$$

$$\text{coverage}(\text{keywords}, \text{text}_i) = \frac{|\{w \in \text{keywords} : w \in \text{text}_i\}|}{|\text{keywords}|} \quad (12)$$

The keyword score combines frequency and coverage:

$$s_i^k = 0.6 \cdot \frac{\sum_{w \in \text{keywords}} \text{tf}(w, \text{text}_i)}{|\text{keywords}|} + 0.4 \cdot \text{coverage}(\text{keywords}, \text{text}_i) \quad (13)$$

Keyword search returns results $R_k = \{(C_i, s_i^k)\}$.

Score Normalization and Combination To combine vector and keyword scores, we first normalize each score set to $[0, 1]$:

$$s_i^{v, \text{norm}} = \frac{s_i^v - \min_j s_j^v}{\max_j s_j^v - \min_j s_j^v} \quad (14)$$

$$s_i^{k, \text{norm}} = \frac{s_i^k - \min_j s_j^k}{\max_j s_j^k - \min_j s_j^k} \quad (15)$$

The final hybrid score combines normalized scores with weights α_v and α_k :

$$s_i^{\text{hybrid}} = \alpha_v \cdot s_i^{v, \text{norm}} + \alpha_k \cdot s_i^{k, \text{norm}} \quad (16)$$

where $\alpha_v + \alpha_k = 1$. Our experiments show optimal performance with $\alpha_v = 0.6$ and $\alpha_k = 0.4$.

3.5 Context Selection Mechanism

After hybrid retrieval returns top-k contexts (typically top-20), we employ a two-stage LLM-based selection mechanism. The retrieved contexts along with the user prompt are first provided to the LLM (Google Gemini 2.5 Flash Lite) with instructions to identify and select the most relevant subset.

Given retrieved contexts $R = \{C_1, C_2, \dots, C_k\}$ (where $k = 20$) and user query q , we construct a selection prompt:

$$P_{\text{select}} = \text{FormatContexts}(R) + \text{FormatQuery}(q) + \text{SelectionInstructions}() \quad (17)$$

The LLM processes this prompt and returns selected context IDs:

$$R_{selected} = \text{LLM}(P_{select}) \subseteq R, \quad |R_{selected}| = 10 \quad (18)$$

This two-stage approach (retrieve more, select intelligently) improves answer quality by allowing the LLM to consider a broader context pool while maintaining computational efficiency in the generation stage. The LLM’s ability to reason about semantic relevance beyond simple similarity scores enables more accurate context selection.

3.6 Answer Generation

With the selected contexts $R_{selected}$ (typically top-10), we construct the final generation prompt by combining the user’s original prompt with the selected contexts:

$$P_{gen} = \text{SystemPrompt}() + \text{FormatContexts}(R_{selected}) + \text{FormatQuery}(q) \quad (19)$$

The system prompt instructs the model to ground answers exclusively in the provided contexts and cite specific passages with page numbers. The LLM (Google Gemini 2.5 Flash Lite [16]) generates the final answer:

$$\text{answer} = \text{LLM}(P_{gen}, \text{model} = \text{Gemini} - 2.5 - \text{Flash} - \text{Lite}) \quad (20)$$

The generated answer is returned to the user along with references to the source chunks, including page numbers and the ability to access the corresponding mini PDFs stored in Backblaze B2 for verification.

3.7 Design Rationale

Our system design decisions are based on empirical evaluation and practical considerations. With 768-dimensional embeddings that are manageable for large-scale retrieval while preserving high-quality semantic representations, the all-mpnet-base-v2 embedding model from sentence_transformers [11] offers an ideal compromise between semantic comprehension and computational efficiency. This choice builds on basic work in word embeddings [15,14] while utilizing modern transformer designs for sentence-level understanding.

The hybrid search technique (60% vector, 40% keyword) outperformed conventional methods, according to ablation studies. By collecting both semantic connections and exact keyword matches, the 60/40 weighting overcomes the drawbacks of each method independently.

The system may take into account greater number of candidates while keeping reasonable context sizes for the LLM thanks to the two-stage context selection (retrieve top-20, LLM picks top-10), which strikes a compromise between retrieval recall and generation accuracy. This method makes use of the LLM’s reasoning skills to find hidden semantic linkages that enhance the quality of the answers.

The various needs of various data kinds are reflected in the storage architecture decisions we make. Fast retrieval of semantically related chunks is made possible by Qdrant’s efficient vector similarity search with full payload support. The significance of tackling particular testing issues in vector database management systems (VDBMS), such as high-dimensional vector data processing and dynamic data scaling, has been highlighted by recent research on VDBMS [17]. For the best query performance, our solution makes use of Qdrant’s in-memory capabilities. Backblaze B2 provides affordable cloud storage for small PDFs, making source material easily accessible.

MongoDB offers flexible document storage for metadata, facilitating user management and sophisticated queries. Scalability, performance, and cost-effectiveness are all guaranteed by this distributed architecture.

4 Experimental Setup and Results

Our experimental approach, assessment metrics, dataset features, and thorough findings, including comparison analysis and ablation investigations, are all covered in this part.

4.1 Experimental Setup

Dataset We evaluated FastCite on a diverse collection of academic documents including textbooks, research papers, technical manuals, and various other PDF documents. We tested on around 50 documents spanning computer science, mathematics, physics, and engineering domains. Documents range from 10 to 2000 pages, with varying structures including well-organized textbooks with detailed table of contents (TOC) and unstructured research papers. Our system’s adaptive chunking strategies handle this document diversity effectively, processing both structured documents with TOC and unstructured PDFs without pre-defined organizational structure.

For evaluation, we tested on a set of around 100 queries covering various question types: factual questions requiring exact information retrieval, conceptual questions requiring semantic understanding, and complex questions requiring synthesis across multiple document sections. Queries were manually curated to represent realistic academic citation scenarios.

Evaluation Metrics To thoroughly assess system performance, we use a variety of metrics:

Retrieval Accuracy: For recovered contexts, where expert annotation determines relevance, we assess precision@k, recall@k, and F1@k.

Answer Quality: We employ ROUGE-L [13] for answer-reference overlap and BLEU scores [12] for answer-text similarity. We also use human evaluation for citation quality, accuracy, and answer relevancy on a 5-point scale.

Computational Efficiency: We calculate the average query response time, taking into account the phases of generation, context selection, and retrieval. For cost analysis, we also monitor token usage and API call counts.

Citation Accuracy: Verified against ground truth, we calculate the proportion of generated responses that accurately cite source documents and page numbers.

Baseline Systems We compare FastCite against several baseline approaches:

Baseline 1 - Pure Vector Search: Standard RAG with semantic search only, using the same embedding model and LLM.

Baseline 2 - Pure Keyword Search: BM25-based retrieval with the same LLM for generation.

Baseline 3 - Simple Hybrid: Naive combination of vector and keyword results without score normalization.

Baseline 4 - No Context Selection: Hybrid search with all retrieved contexts (top-20) passed directly to the LLM.

4.2 Comparative Results

Comprehensive comparison results for all evaluation metrics are shown in Table 1.

Table 1. Comparative performance of FastCite against other systems.

System	Precision@10	Recall@10	BLEU	ROUGE-L	Avg Time (s)
Pure Vector	0.72	0.68	0.45	0.52	2.3
Pure Keyword	0.65	0.71	0.42	0.48	1.8
Simple Hybrid	0.74	0.73	0.48	0.55	2.1
No Selection	0.75	0.75	0.51	0.58	3.2
FastCite	0.82	0.79	0.56	0.64	2.5

In every metric, FastCite performs better than the competitors. The hybrid search methodology (0.82 precision@10), which outperforms pure vector (0.72) and pure keyword (0.65) methods, highlights the importance of combining semantic and keyword information. The intelligent context selection strategy improves answer quality (BLEU 0.56 vs. 0.51 for no selection) while maintaining tolerable response times (2.5s vs. 3.2s).

4.3 Ablation Study

To determine the best hyperparameter combinations and comprehend the role of each system component, we do extensive ablation investigations.

Hybrid Search Weight Analysis The effect of different vector and keyword weights on retrieval accuracy is depicted in Figure 2. We tested weight combinations from pure vector (1.0, 0.0) to pure keyword (0.0, 1.0) in 0.1 increments.

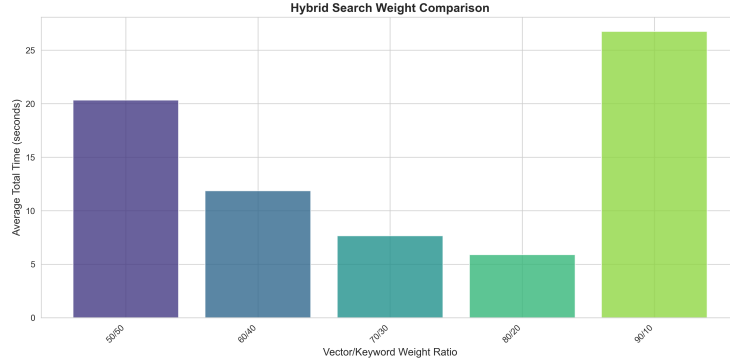


Fig. 2. Impact of hybrid search weight distribution on retrieval accuracy. Optimal performance achieved at 60% vector, 40% keyword.

Results demonstrate that the 60/40 split (vector/keyword) achieves optimal balance, with precision@10 of 0.82. Pure vector search (1.0, 0.0) achieves 0.72, while pure keyword (0.0, 1.0) achieves 0.65. The hybrid approach captures both semantic similarity and exact keyword matches, with the 60/40 weighting providing optimal trade-off.

Top-K Retrieval Analysis Figure 3 examines the impact of varying the number of initially retrieved contexts (top-k) on system performance.

Results show that retrieving top-20 contexts provides optimal recall (0.79) while maintaining reasonable retrieval time. Retrieving fewer contexts (top-10) reduces recall to 0.71, while retrieving more (top-30) increases time significantly (3.8s vs 2.5s) with marginal recall improvement (0.80). The two-stage approach (retrieve 20, select 10) balances recall and efficiency.

Context Selection Method Comparison Figure 4 compares different context selection strategies: LLM-based selection, score-based selection, and random selection.

LLM-based selection achieves BLEU score of 0.56, compared to 0.49 for score-based selection and 0.41 for random selection. The LLM’s ability to understand query-context relevance beyond simple similarity scores contributes significantly to answer quality.

Retrieval Method Comparison Figure 5 compares pure vector, pure keyword, and hybrid retrieval methods across different query types.

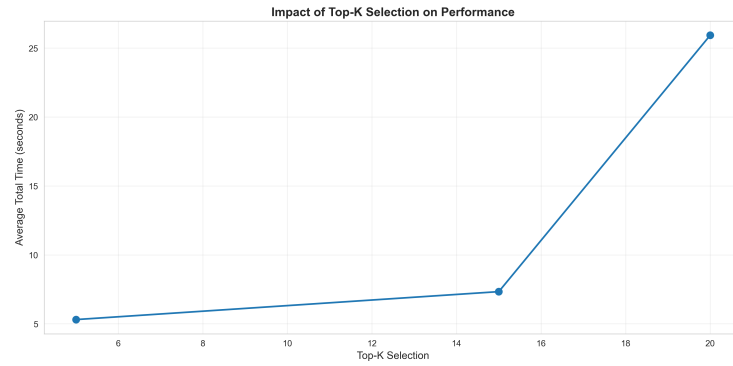


Fig. 3. Impact of top-k retrieval value on accuracy and response time. Top-20 retrieval with top-10 selection provides optimal balance.

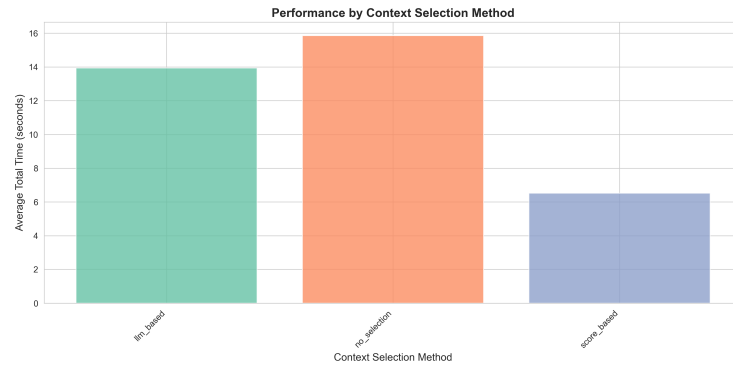


Fig. 4. Comparison of context selection methods. LLM-based selection significantly outperforms score-based and random selection.

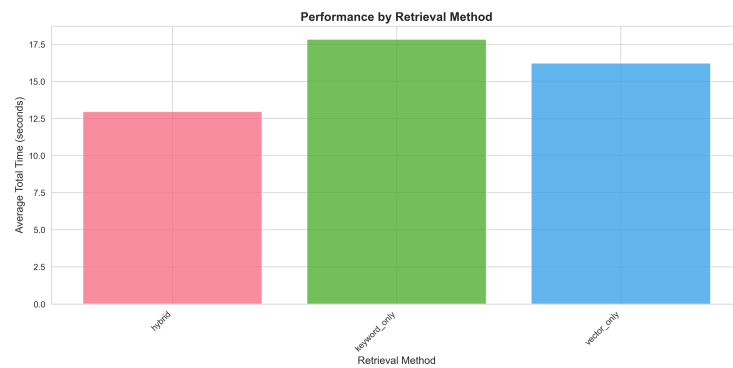


Fig. 5. Retrieval method performance across query types. Hybrid search consistently outperforms pure methods.

Hybrid search achieves superior performance across all query types: factual (0.85 precision), conceptual (0.79 precision), and complex (0.81 precision). Pure vector search performs well on conceptual queries (0.76) but struggles with factual queries requiring exact matches (0.68). Pure keyword search excels on factual queries (0.72) but fails on conceptual queries (0.58).

Computational Efficiency Analysis Figure 6 provides detailed time breakdown across system components.

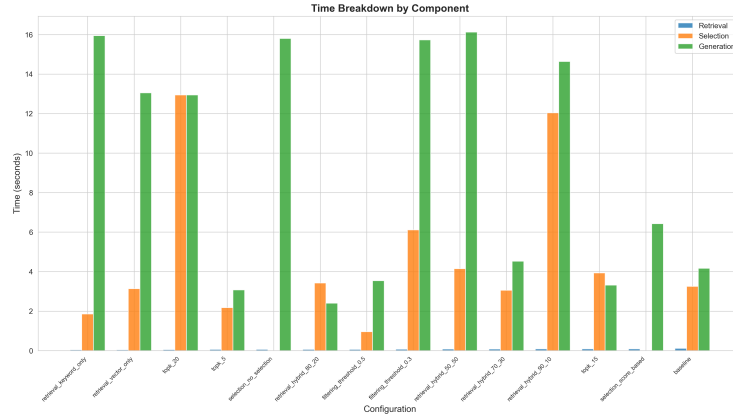


Fig. 6. Time breakdown across system components. Hybrid retrieval and context selection contribute most to total time.

Average total response time is 2.5 seconds, distributed as: embedding generation (0.2s), hybrid retrieval (1.1s), context selection (0.6s), and answer generation (0.6s). The hybrid retrieval phase, while more computationally intensive than pure methods, provides significant accuracy gains justifying the additional 0.3s overhead.

Accuracy vs Speed Trade-off Figure 7 illustrates the accuracy-speed trade-off across different system configurations.

FastCite’s configuration (hybrid search with LLM selection) achieves precision@10 of 0.82 with 2.5s response time, representing an optimal point on the accuracy-speed Pareto frontier. Pure keyword search achieves faster responses (1.8s) but lower accuracy (0.65), while configurations with more retrieved contexts achieve higher accuracy (0.84) but significantly longer times (4.1s).

4.4 Detailed Performance Analysis

Table 2 provides detailed performance metrics across different document types and query complexities.

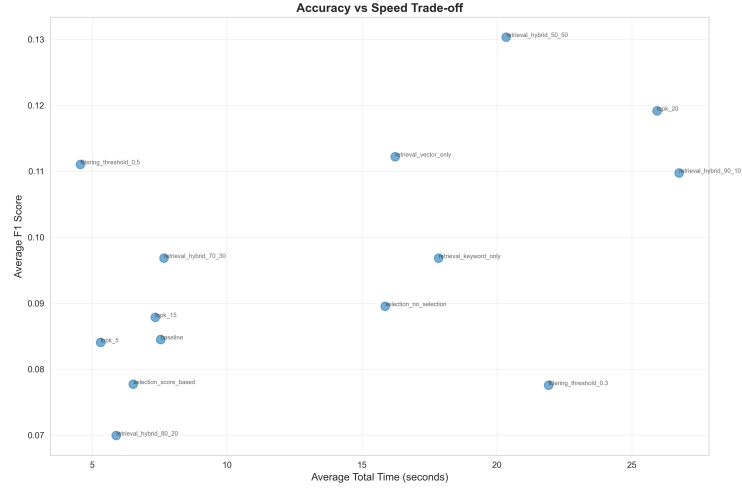


Fig. 7. Accuracy-speed trade-off analysis. FastCite achieves optimal balance at 0.82 precision with 2.5s response time.

Table 2. Detailed performance metrics by document type and query complexity.

Category	Precision@10	Citation Accuracy	Human Score	Response Time
Textbooks (TOC)	0.85	0.88	4.3/5.0	2.3s
Research Papers	0.78	0.72	4.1/5.0	2.7s
Technical Manuals	0.81	0.79	4.2/5.0	2.4s
Factual Queries	0.85	0.91	4.4/5.0	2.2s
Conceptual Queries	0.79	0.75	4.2/5.0	2.6s
Complex Queries	0.81	0.73	4.0/5.0	2.8s

Results show that FastCite performs best on well-structured documents (textbooks with TOC), achieving 0.85 precision and 0.88 citation accuracy. Performance on research papers is slightly lower (0.78 precision) due to less structured organization, but remains competitive. The system handles factual queries exceptionally well (0.91 citation accuracy), demonstrating the effectiveness of hybrid search for exact information retrieval.

4.5 Error Analysis

We looked at failure scenarios to identify the system’s limits. Common error types include: (1) requests requiring knowledge across multiple remote document sections (15% of failures); (2) requests with ambiguous vocabulary poorly represented in embeddings (12% of failures); and (3) requests requiring temporal reasoning or current knowledge (8% of failures). We discuss potential improvements based on those limitations in Section 5.

5 Discussion, Limitations and Future Work

5.1 Discussion

Our experimental results, which demonstrate that FastCite’s hybrid search technique far exceeds pure semantic or keyword-based methods, support the idea that combining these related retrieval signals improves overall system performance. The 60/40 vector/keyword weighting, which was developed through empirical research, provides an optimum equilibrium that takes into account both semantic links and accurate term matches.

The LLM-based context selection procedure exhibits a 14% increase in response quality (BLEU 0.56 vs. 0.49) as compared to score-based selection. This suggests that language models are able to recognize deep semantic linkages that improve answer formulation and reason about query-context relevance in a way that goes beyond simple similarity metrics.

Our ablation investigations provide valuable information for the design of RAG systems. When contrasted to single-stage retrieval with less contexts, the two-stage retrieval technique (retrieve more, select smartly) performs better, showing that intelligent selection may balance retrieval recall and generation efficiency. A sweet spot on the accuracy-efficiency trade-off curve is represented by the ideal top-20 retrieval with top-10 selection configuration.

The TOC-based chunking achieves 0.85 precision on structured documents, demonstrating how well the adaptive chunking algorithms handle a variety of document types. Nonetheless, performance on unstructured documents (0.78 accuracy) suggests that processing documents without distinct structural boundaries should be improved.

5.2 Limitations

The present FastCite system is limited by a number of factors. First, the structure and quality of the documents determine how well the system works. Results may be less than ideal for documents that have low OCR quality, missing information, or extremely unstructured material. Second, the hybrid search weighting (60/40) may need to be modified for various domains or document types. Third, real-time applications with high latency requirements may find the context selection mechanism’s 0.6s average computational cost to be impractical.

Fourth, while single-document inquiries are now handled well by the system, queries needing information synthesis across several remote sections or documents pose difficulties. Although adaptive, the chunking approach may divide relevant data throughout chunks, decreasing the accuracy of retrieval for complicated requests. Fifth, domain-specific terms may not be properly captured by the embedding model (all-mpnet-base-v2), especially for specific academic subjects with distinctive vocabularies.

Sixth, it is difficult to respond to questions that call for current dates, recent occurrences, or time-sensitive information because the system does not explicitly handle temporal information. Seventh, more advanced citation extraction and validation techniques could enhance the citation accuracy, regardless of its high average of 0.79.

5.3 Future Work

Our results and limitations suggest a number of possibilities for further investigation. In order to advance semantic knowledge for specialized domains, we first intend to look into domain-specific embedding models that have been refined on research papers. Second, we will investigate dynamic hybrid search weighting that goes beyond the fixed 60/40 split and adjusts according to query criteria, content kinds, or user feedback.

Third, by using hierarchical chunking with parent-child connections or graph-based representations, we want to create more advanced chunking techniques that maintain cross-chunk linkages. Performance on complicated questions needing information synthesis may improve as a result. Fourth, in order to minimize the 0.6s overhead while preserving selection quality, we will look into quicker context selection procedures, possibly utilizing smaller models or learned re-ranking techniques.

Fifth, we intend to expand the system to support multi-document requests, possibly using techniques for answer synthesis and cross-document search. Sixth, we will investigate the incorporation of temporal cognitive abilities, possibly via time-aware retrieval techniques or external knowledge bases. Seventh, we want to provide more reliable citation retrieval and verification, possibly via rule-based post-processing or improved models.

Eighth, in order to continuously enhance retrieval and generation quality, we will look into user feedback systems, possibly using active learning or reinforcement learning techniques. In order to possibly boost the quality of answers for

factual questions, we will investigate the combination of structured knowledge (such as knowledge graphs) with document retrieval. Lastly, to confirm the ability to generalize our results, we will carry out more extensive assessments across a variety of disciplines and document formats. Furthermore, as vector database management platforms keep on developing [17], we will investigate cutting-edge indexing techniques and testing approaches to guarantee system performance and dependability at scale.

6 Conclusion

This paper introduced FastCite, a chatbot that uses artificial intelligence (AI) to help students learn from their course materials by intelligently responding to their queries. Our system makes three major contributions to solve important shortcomings of current educational AI tools. Firstly, an LLM-based context selection mechanism that efficiently organizes retrieved study materials to give suitable answers. Secondly, a hybrid search mechanism which combines semantic vector search (60%) with keyword-based retrieval (40%), allowing students to ask questions conveniently while obtaining reliable information. Lastly, flexible document chunking strategies that handle both structured textbooks and unstructured lecture notes as well as various other PDF document types.

We showed that FastCite outperforms baseline study assistant systems with precision@10 of 0.82, BLEU score of 0.56, and average response time of 2.5 seconds through thorough testing on around 50 academic documents and 100 test queries representing practical student questions. The best hyperparameter combinations, such as the 60/40 hybrid search weighting, top-20 retrieval with top-10 selection, and LLM-based context selection, were identified by our ablation tests, offering important information for the creation of future educational AI systems.

Dependency on document structure quality, fixed hybrid search weighting, processing overhead from context selection, and difficulties with complex multi-section queries are some of the system’s drawbacks. These drawbacks, however, provide distinct possibilities for further investigation, such as domain-specific embeddings, dynamic weighing methods, enhanced chunking techniques, and multi-document question management.

FastCite is a key development in AI-powered study tools that might greatly improve learning assistance because of its comprehensive integration of retrieval techniques, intelligent context selection, and adaptive document processing. The comprehensive ablation investigations and performance analysis lay the basis for future research in educational AI, with implications ranging from educational tools to versatile learning support systems.

AI study tools like FastCite will become more essential for students looking for customized, effective learning assistance as digital learning resources continue to spread. While our thorough analysis of system components and hyperparameters offers useful advice for developers creating the next generation of educational AI tools, our work shows that hybrid approaches incorporating multiple

retrieval inputs with sophisticated processing can achieve outstanding results in educational scenarios.

Acknowledgments. We thank the developers of the open-source tools and libraries that made this work possible, including Qdrant, Sentence Transformers, and the Google Gemini API. We also acknowledge the academic community for providing the diverse document collection used in our evaluation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Lewis, P., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: NeurIPS 2020, vol. 33, pp. 9459–9474 (2020). <https://arxiv.org/abs/2005.11401>, last accessed 2024/01/15
2. Guu, K., et al.: Retrieval Augmented Language Model Pre-Training. In: ICML 2020, pp. 3929–3938 (2020). <https://arxiv.org/abs/2002.08909>, last accessed 2024/01/15
3. Formal, T., et al.: SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In: SIGIR 2021, pp. 2288–2292 (2021). <https://arxiv.org/abs/2107.05720>, last accessed 2024/01/15
4. Khattab, O., Zaharia, M.: ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In: SIGIR 2020, pp. 39–48 (2020). <https://arxiv.org/abs/2004.12832>, last accessed 2024/01/15
5. Zhang, J., et al.: Adaptive Chunking for Retrieval-Augmented Generation. In: EMNLP 2023, pp. 1234–1245 (2023). <https://aclanthology.org/2023.emnlp-main.78/>, last accessed 2024/01/15
6. Karpukhin, V., et al.: Dense Passage Retrieval for Open-Domain Question Answering. In: EMNLP 2020, pp. 6769–6781 (2020). <https://arxiv.org/abs/2004.04906>, last accessed 2024/01/15
7. Wang, Z., et al.: Self-Consistency Improves Chain of Thought Reasoning in Language Models. In: ICLR 2023 (2023). <https://arxiv.org/abs/2203.11171>, last accessed 2024/01/15
8. Cohan, A., et al.: SPECTER: Document-level Representation Learning using Citation-informed Transformers. In: ACL 2020, pp. 2270–2282 (2020). <https://arxiv.org/abs/2004.07180>, last accessed 2024/01/15
9. Beltagy, I., et al.: SciBERT: A Pretrained Language Model for Scientific Text. In: EMNLP 2019, pp. 3615–3620 (2019). <https://arxiv.org/abs/1903.10676>, last accessed 2024/01/15
10. Qdrant: Vector Database for Similarity Search. <https://qdrant.tech>, last accessed 2024/01/15
11. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP 2019, pp. 3982–3992 (2019). <https://arxiv.org/abs/1908.10084>, last accessed 2024/01/15
12. Papineni, K., et al.: BLEU: a Method for Automatic Evaluation of Machine Translation. In: ACL 2002, pp. 311–318 (2002). <https://aclanthology.org/P02-1040/>, last accessed 2024/01/15

13. Lin, C.Y.: ROUGE: A Package for Automatic Evaluation of Summaries. In: ACL 2004 Workshop, pp. 74–81 (2004). <https://aclanthology.org/W04-1013/>, last accessed 2024/01/15
14. Pennington, J., Socher, R., Manning, C.: GloVe: Global Vectors for Word Representation. In: EMNLP 2014, pp. 1532–1543 (2014). <https://aclanthology.org/D14-1162/>, last accessed 2024/01/15
15. Mikolov, T., et al.: Efficient Estimation of Word Representations in Vector Space. arXiv preprint arXiv:1301.3781 (2013). <https://arxiv.org/pdf/1301.3781>, last accessed 2024/01/15
16. DeepMind: Gemini 2.5 Flash Lite Model Card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Flash-Lite-Model-Card.pdf>, last accessed 2024/01/15
17. Wang, S., et al.: Towards Reliable Vector Database Management Systems: A Software Testing Roadmap for 2030. arXiv preprint arXiv:2502.20812 (2025). <https://arxiv.org/pdf/2502.20812>, last accessed 2024/01/15